CU7 Performance Assessment — LangganKu (starter)

Fill in your name and the last four digits of your IC, then work through the Tasks of the paper **in this notebook**. First create your own data file in a terminal: `python make_my_data.py <L4>` (the file `subscriber_churn.csv` appears in this folder). Keep the notebook runnable from top to bottom.

In [1]:
```
import warnings; warnings.filterwarnings("ignore")
import mlflow, numpy as np, pandas as pd

NAME = "MOHAMMAD RAMDZUANNY BIN MUSRAM RAKUNMAN"      # e.g. "AISYAH BINTI AHMAD"
L4   = "6645"                # last four digits of your IC, keep the leading zero

# ---- your personal values (rules in the paper) ----
SEED      = int(L4)                                  # 6645 -> random_state for the split and the random forest
TEST_SIZE = 0.20 if int(L4[3]) % 2 == 0 else 0.25    # 4th digit 5 is odd -> 0.25
DEPTH     = 3 + int(L4[1]) % 4                       # 3 + (6 % 4) = 5 -> max_depth of the random forest
C1        = round(0.04 * (int(L4[2]) + 1), 2)        # 0.04 * (4 + 1) = 0.2 -> C of logreg-c<C1>; round() keeps 2 d
C2        = 10 * C1
RANK      = "metrics.test_recall"
PORT      = 5100 + int(L4[2:])                       # 5100 + 45 = 5145 -> port for mlflow models serve (Task 5)
EXP       = f"LangganKu_Churn_{L4}"             # experiment name
REG       = f"langganku_churn_{L4}"             # registered model name
print(SEED, TEST_SIZE, DEPTH, C1,C2,RANK, PORT, EXP, REG)
```

```
6645 0.25 5 0.2 2.0 metrics.test_recall 5145 LangganKu_Churn_6645 langganku_churn_6645
```

Task 1 — Set-up, split and pipeline

In [2]:
```
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
```

```
# point MLflow at your own SQLite store; the .db file is created on first use
mlflow.set_tracking_uri(f"sqlite:///langganku_{L4}.db")
# select the experiment; MLflow creates it if it does not exist yet
mlflow.set_experiment(EXP)

# load the data file made by make_my_data.py and inspect it
df = pd.read_csv("subscriber_churn.csv")
print(df.shape)          # (rows, columns)
print(df.dtypes)         # type of each column
print(df.isna().sum())   # number of missing values per column
print(df["churned"].value_counts(normalize=True))   # share of each class (0 = repaid, 1 = churned)

# features = every column except the ID and the target
X = df.drop(columns=["subscriber_id", "churned"])
y = df["churned"]
# same SEED and TEST_SIZE everywhere, so the split is reproducible
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=TEST_SIZE, random_state=SEED)

# text columns (the four categorical ones) get one-hot encoded; numeric columns pass through unchanged
cat_cols = X.select_dtypes(exclude="number").columns.tolist()
# handle_unknown="ignore": a category not seen in training becomes all zeros instead of raising an error
prep = ColumnTransformer([("cat", OneHotEncoder(handle_unknown="ignore"), cat_cols)], remainder="passthrough")
# one pipeline = pre-processing + model. The encoder is fitted on the training data only (no leakage)
# and is saved together with the model. Only the "model" step changes between candidates.
pipe = Pipeline([("prep", prep), ("model", LogisticRegression(max_iter=1000))])

# returns the four classification metrics as a dict
def get_metrics(y_true, y_pred):
    return {"accuracy": accuracy_score(y_true, y_pred),
            "precision": precision_score(y_true, y_pred, zero_division=0),   # 0 instead of a warning if no 1s are
            "recall": recall_score(y_true, y_pred),
            "f1": f1_score(y_true, y_pred)}

# fit the baseline on the training set and score it on the test set
pipe.fit(X_train, y_train)
print("baseline test metrics:", get_metrics(y_test, pipe.predict(X_test)))
```

```
2026/10/20 11:48:59 INFO mlflow.store.db.utils: Creating initial MLflow database tables...
2026/10/20 11:48:59 INFO mlflow.store.db.utils: Updating database tables
2026/10/20 11:48:59 INFO mlflow.tracking.fluent: Experiment with name 'LangganKu_Churn_6645' does not exist. Creatin
g a new experiment.
```

```
(1200, 10)
subscriber_id      object
avg_hours_week    float64
plan               object
main_device        object
tenure_months       int64
monthly_fee_rm    float64
support_calls       int64
payment_method     object
contract           object
churned             int64
dtype: object
subscriber_id     0
avg_hours_week    0
plan              0
main_device       0
tenure_months     0
monthly_fee_rm    0
support_calls     0
payment_method    0
contract          0
churned           0
dtype: int64
churned
0    0.791667
1    0.208333
Name: proportion, dtype: float64
baseline test metrics: {'accuracy': 0.8166666666666667, 'precision': 0.5909090909090909, 'recall': 0.412698412698412
7, 'f1': 0.48598130841121495}
```

Task 2 — Track the candidates in ONE experiment

In [3]:
```python
import mlflow.sklearn
from sklearn.ensemble import RandomForestClassifier

# one entry per candidate, in the order the paper requires:
# (run name without L4, model, model settings to log as params, value of the model_type tag)
# f"{C1:g}" writes the number without trailing zeros, e.g. 0.2 -> "0.2", 4.0 -> "4"
candidates = [
```

```
    ("logreg-base", LogisticRegression(max_iter=1000), {"max_iter": 1000}, "linear"),
    (f"logreg-c{C1:g}", LogisticRegression(C=C1, max_iter=1000), {"C": C1, "max_iter": 1000}, "linear"),
    (f"logreg-c{C2:g}", LogisticRegression(C=C2, max_iter=1000), {"C": C2, "max_iter": 1000}, "linear"),
    (f"rf-d{DEPTH}", RandomForestClassifier(n_estimators=100, max_depth=DEPTH, random_state=SEED),
     {"n_estimators": 100, "max_depth": DEPTH}, "tree_ensemble"),
]

model_uris = {}   # run_id -> URI of the logged model, used in Task 4 to register models
for name, model, params, mtype in candidates:
    # one run per candidate, inside the experiment selected in Task 1
    with mlflow.start_run(run_name=f"{name}-{L4}") as run:
        # swap the model step of the same pipeline, then fit on the training set
        pipe.set_params(model=model).fit(X_train, y_train)
        # params: model name, split settings and the model's own settings (each name logged only once)
        mlflow.log_params({"model": type(model).__name__, "test_size": TEST_SIZE, "split_seed": SEED, **params})
        mlflow.set_tag("model_type", mtype)
        # metrics: the same get_metrics() on train and test, keys prefixed with train_ / test_
        mlflow.log_metrics({f"train_{k}": v for k, v in get_metrics(y_train, pipe.predict(X_train)).items()})
        mlflow.log_metrics({f"test_{k}": v for k, v in get_metrics(y_test, pipe.predict(X_test)).items()})
        # log the whole pipeline. input_example lets MLflow record the input schema (column names and types),
        # which the served model later uses to check requests. cloudpickle is required by the paper.
        info = mlflow.sklearn.log_model(pipe, name="model", input_example=X_train.head(3),
                                        serialization_format="cloudpickle")
        # keep the logged model's URI so Task 4 can register it
        model_uris[run.info.run_id] = info.model_uri
        print("logged", f"{name}-{L4}")
```

```
2026/10/20 11:49:00 WARNING mlflow.sklearn: Saving scikit-learn models in the pickle or cloudpickle format requires
exercising caution because these formats rely on Python's object serialization mechanism, which can execute arbitrar
y code during deserialization. The recommended safe alternative is the 'skops' format. For more information, see: ht
tps://scikit-learn.org/stable/model_persistence.html
2026/10/20 11:49:02 WARNING mlflow.sklearn: Saving scikit-learn models in the pickle or cloudpickle format requires
exercising caution because these formats rely on Python's object serialization mechanism, which can execute arbitrar
y code during deserialization. The recommended safe alternative is the 'skops' format. For more information, see: ht
tps://scikit-learn.org/stable/model_persistence.html
logged logreg-base-6645
2026/10/20 11:49:03 WARNING mlflow.sklearn: Saving scikit-learn models in the pickle or cloudpickle format requires
exercising caution because these formats rely on Python's object serialization mechanism, which can execute arbitrar
y code during deserialization. The recommended safe alternative is the 'skops' format. For more information, see: ht
tps://scikit-learn.org/stable/model_persistence.html
```

```
logged logreg-c0.2-6645
2026/10/20 11:49:05 WARNING mlflow.sklearn: Saving scikit-learn models in the pickle or cloudpickle format requires exercising caution because these formats rely on Python's object serialization mechanism, which can execute arbitrary code during deserialization. The recommended safe alternative is the 'skops' format. For more information, see: https://scikit-learn.org/stable/model_persistence.html
logged logreg-c2-6645
logged rf-d5-6645
```

Task 3 — Choose the best run from code and justify it

In [4]:
```python
# all runs of the experiment as a DataFrame, sorted by test_f1 (highest first)
# column names: metrics are "metrics.<name>", params "params.<name>", the run name is the tag "tags.mlflow.runName"
runs = mlflow.search_runs(experiment_names=[EXP], order_by=[f"{RANK} DESC"])
print(runs[["tags.mlflow.runName", "metrics.train_f1", "metrics.test_f1",RANK, "metrics.test_accuracy",]])
for x in runs["metrics.train_f1"]:
   float(x)
for y in runs["metrics.test_f1"]:
    print (f"overfit_gap : {abs(float(y) - float(x))}")
# after sorting, row 0 is the best run
best_run_id = runs.loc[2, "run_id"]
challenger_id = runs.loc[1, "run_id"]
best_name = runs.loc[2, "tags.mlflow.runName"]
print("best run:", best_name)
```

```
  tags.mlflow.runName  metrics.train_f1  metrics.test_f1  metrics.test_recall  \
0      logreg-c2-6645          0.573248         0.485981             0.412698
1    logreg-base-6645          0.570513         0.485981             0.412698
2    logreg-c0.2-6645          0.553333         0.480769             0.396825
3          rf-d5-6645          0.393162         0.270270             0.158730

   metrics.test_accuracy
0               0.816667
1               0.816667
2               0.820000
3               0.820000
overfit_gap : 0.0928189152488218
overfit_gap : 0.0928189152488218
overfit_gap : 0.08760683760683763
overfit_gap : 0.12289212289212287
best run: logreg-c0.2-6645
```

Why I chose this run (2–3 sentences):

I chose logreg-c0.2-6645 because it has the lowest overfit_gap (0.08760683760683763). It is roughly 0.5 points lower than logreg-base-6645 and logreg-c2-6645 (0.09281), but clearly better than rf-d5-6645 (0.12289212289212287). All runs show overfitting even when the other metrics are showing differences, hence the decision to choose champion is based solely on the overfit gap

Task 4 — Register, alias and roll back

In [5]:
```
row_number_example = int(L4)%len(df)
row_number_example
```

Out[5]: 645

In [6]:
```
df.iloc[645]
```

Out[6]:
```
subscriber_id             S0798
avg_hours_week             3.0
plan                   Standard
main_device                  TV
tenure_months                14
monthly_fee_rm             42.9
support_calls                 1
payment_method    Bank transfer
contract                6-month
churned                       1
Name: 645, dtype: object
```

In [7]:
```
from mlflow import MlflowClient
client = MlflowClient()   # uses the tracking URI set in Task 1

# prints which version the alias "champion" points to
def show_champion(step):
    print(step, "-> champion is version", client.get_model_version_by_alias(REG, "champion").version)

def show_challenger(step):
    print(step, "-> challenger is version", client.get_model_version_by_alias(REG, "challenger").version)

# version 1 = the baseline run; each register_model() call on the same name creates the next version
base_id = runs.loc[runs["tags.mlflow.runName"] == f"logreg-base-{L4}", "run_id"].iloc[0]
v1 = mlflow.register_model(model_uris[base_id], REG)
client.set_registered_model_alias(REG, "champion", v1.version)
show_champion("registered v1")

# version 2 = the best run from Task 3 (registered even if it is the same run as the baseline)
v2 = mlflow.register_model(model_uris[best_run_id], REG)
client.set_registered_model_alias(REG, "champion", v2.version)   # an alias points to one version, so this moves it
client.set_model_version_tag(REG, v2.version, "approved_by", NAME)
client.update_model_version(REG, v2.version, description=f"{best_name}: lowest overfit gap of all candidates")
show_champion("registered v2")

challenger = mlflow.register_model(model_uris[challenger_id], REG)
client.set_registered_model_alias(REG, "challenger", challenger.version)
show_challenger("challenger v1")

# roll-back: champion -> version 1, then roll-forward: champion -> version 2
```

```
client.set_registered_model_alias(REG, "champion", v1.version)
show_champion("roll-back")
client.set_registered_model_alias(REG, "champion", v2.version)
show_champion("roll-forward")

# based on df row 645
new_rows = pd.DataFrame({
    "avg_hours_week":[3.0],
    "plan":["Standard"],
    "main_device":["TV"],
    "tenure_months":[14],
    "monthly_fee_rm":[42.9],
    "support_calls":[1],
    "payment_method":["Bank transfer"],
    "contract":["6-month"]
})

# load whichever version the alias points to (version 2 now) and predict
champion = mlflow.pyfunc.load_model(f"models:/{REG}@champion")
print(new_rows.assign(predicted_class=champion.predict(new_rows)))   # 0 = stays, 1 = churns
```

```
registered v1 -> champion is version 1
registered v2 -> champion is version 2
challenger v1 -> challenger is version 3
roll-back -> champion is version 1
roll-forward -> champion is version 2
   avg_hours_week      plan main_device  tenure_months  monthly_fee_rm  \
0             3.0  Standard          TV             14            42.9

   support_calls payment_method contract  predicted_class
0              1  Bank transfer  6-month                0
```

```
Successfully registered model 'langganku_churn_6645'.
Created version '1' of model 'langganku_churn_6645'.
Registered model 'langganku_churn_6645' already exists. Creating a new version of this model...
Created version '2' of model 'langganku_churn_6645'.
Registered model 'langganku_churn_6645' already exists. Creating a new version of this model...
Created version '3' of model 'langganku_churn_6645'.
```

Task 5 — Serve the champion and call it

In [8]:
```
import os, sys, json, time, subprocess, requests

# copy of the current environment variables plus MLFLOW_TRACKING_URI, so the server reads your store
env = {**os.environ, "MLFLOW_TRACKING_URI": f"sqlite:///langganku_{L4}.db"}
# start the server in the background; Popen returns immediately and the server keeps running
# --env-manager local: use the current Python environment instead of building a new one
server = subprocess.Popen([sys.executable, "-m", "mlflow", "models", "serve",
                           "-m", f"models:/{REG}@champion", "-p", str(PORT), "--env-manager", "local"], env=env)

URL = f"http://127.0.0.1:{PORT}"
# wait up to ~60 s until the server answers /ping
# note: any server already running on PORT also answers /ping, so the port must be free before this cell
for _ in range(60):
    try:
        if requests.get(f"{URL}/ping").status_code == 200:
            break
    except requests.ConnectionError:   # server not listening yet
        time.sleep(1)

# sends a DataFrame to /invocations in dataframe_split format
# returns (status code, predictions) on success, or (status code, error message) on failure
def predict_via_api(df):
    # to_json -> json.loads turns numpy values into plain Python values that requests can send as JSON
    payload = {"dataframe_split": json.loads(df.to_json(orient="split", index=False))}
    r = requests.post(f"{URL}/invocations", json=payload)
    body = r.json()
    return r.status_code, body.get("predictions", body.get("message"))

status, preds = predict_via_api(new_rows)
print("good request:", status, preds)   # expected: 200 and the same classes as in Task 4

# bad request on purpose: drop a required column -> the schema check fails -> 400 Bad Request
status, message = predict_via_api(new_rows.drop(columns=["support_calls"]))
print("bad request:", status)
print(message[:150])                          # first part of the error message
print(message[message.rfind("Error:"):])      # the reason, at the end of the message
```

```
Downloading artifacts: 100%|██████████| 8/8 [00:00<00:00, 15203.64it/s]
2026/10/20 11:49:07 INFO mlflow.models.flavor_backend_registry: Selected backend for flavor 'python_function'
2026/10/20 11:49:07 INFO mlflow.pyfunc.backend: === Running command 'exec uvicorn --host 127.0.0.1 --port 5145 --wor
kers 1 mlflow.pyfunc.scoring_server.app:app'
INFO:     Started server process [12872]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://127.0.0.1:5145 (Press CTRL+C to quit)
INFO:     127.0.0.1:59642 - "GET /ping HTTP/1.1" 200 OK
INFO:     127.0.0.1:59643 - "POST /invocations HTTP/1.1" 200 OK
good request: 200 [0]
INFO:     127.0.0.1:59644 - "POST /invocations HTTP/1.1" 400 Bad Request
bad request: 400
Failed to predict data '   avg_hours_week      plan main_device  ...  monthly_fee_rm  payment_method contract
0             3.0  Standard          TV
Error: Model is missing inputs ['support_calls'].
2026/10/20 11:49:09 WARNING mlflow.pyfunc.scoring_server: If using `instances` as input key, we internally convert t
he data type from `records` (List[Dict]) type to `list` (Dict[str, List]) type if the data is a pandas dataframe rep
resentation. This might cause schema changes. Please use `inputs` to avoid this conversion.
```

In [11]:
```
server.terminate()
```